VetCare

Trabalho II – Implementação do Sistema VetCare
Etapa 2 · Sprint 1

Disciplina: Engenharia de Software 1
Professora: Dra. Larissa Barbosa Leôncio Pinheiro
Instituição: Universidade Federal da Bahia (UFBA) – Instituto de Computação
Equipe: Guilherme Pimentel Santos Barbosa Gomes e Arthur Pimentel Santos Barbosa Gomes
Repositório: https://github.com/GuilhermePSBG/vetcare
Data: novembro de 2026 · **Versão:** 1.0

Documento complementar ao Documento de Arquitetura (Trabalho I, versão 1.0).

Sumário

1. Objetivo da sprint 3

2. Requisitos implementados na sprint 3

3. Arquitetura implementada 5

4. ADRs: desvios e complementos à Etapa 1 8

5. Implementação 11

6. Telas do sistema em funcionamento 13

7. Testes e verificação 17

8. Como executar e repositório 18

9. Planejamento das próximas sprints 18

1. Objetivo da sprint

Esta é a primeira etapa de implementação do VetCare. O objetivo foi entregar cerca de 30% dos requisitos funcionais do Documento de Requisitos (versão 01.00), em um sistema que funcione de ponta a ponta e que siga a arquitetura definida no Trabalho I: **monolito modular com arquitetura hexagonal (portas e adaptadores) dentro de cada módulo**.

Como a Etapa 1 não fixou a linguagem, a implementação foi feita em **Python**. A escolha do framework e os pontos em que a implementação se afastou da Etapa 1 estão registrados como ADRs na seção 4.

> **Resultado:** 6 dos 17 requisitos funcionais implementados (35%), todos de prioridade Alta; 4 módulos da arquitetura em funcionamento (Identidade e Acesso, Cadastro, Agenda e Auditoria); Portal Web com telas por perfil; 54 testes automáticos, incluindo testes que verificam a regra de dependência entre módulos.

2. Requisitos implementados na sprint

Foram selecionados **6 de 17 requisitos (35%)**, próximo dos 30% pedidos. A seleção seguiu três critérios:

- **Prioridade:** somente requisitos de prioridade Alta.
- **Dependência:** quase todos os outros requisitos dependem destes. Não há atendimento, prontuário ou pagamento sem usuário autenticado, tutor, animal, veterinário e consulta.
- **Fatia vertical completa:** a sprint entrega um fluxo útil de ponta a ponta: o administrador cria os usuários, a recepção cadastra tutor e animal e agenda a consulta com um veterinário. Isso exercita as quatro características prioritárias da Etapa 1 (segurança, integridade, manutenibilidade e disponibilidade).

REQ	Requisito	Módulo (Etapa 1)	O que foi implementado (critérios de aceitação)
REQ 01	Manter Usuário	Identidade e Acesso	Inclusão, alteração, exclusão e consulta de usuários com perfil (Administrador, Recepcionista, Veterinário). E-mail único; senha com 8+ caracteres, letras e números, guardada só como hash; o sistema sempre mantém um administrador ativo; ninguém exclui ou desativa a si mesmo. Busca e paginação.
REQ 02	Autenticar Usuário	Identidade e Acesso	Login por e-mail e senha com token assinado (validade de 8 h). Mensagem genérica para e-mail ou senha errados; bloqueio de 15 min após 5 tentativas falhas; usuário desativado perde o acesso mesmo com token válido; menu e telas exibidos conforme o perfil.
REQ 04	Manter Tutor	Cadastro – Tutores e Animais	Cadastro, alteração, consulta e exclusão. CPF validado pelos dígitos verificadores e único; telefone com DDD; e-mail e endereço opcionais. Não exclui tutor que tenha animais.

REQ 05	Manter Animal	Cadastro – Tutores e Animais	Cadastro, alteração, consulta e exclusão, sempre associado a um tutor existente, com nome, espécie, raça, sexo e data de nascimento (não futura). Filtro por tutor. Não exclui animal com consultas registradas.
REQ 06	Manter Veterinário	Cadastro – Equipe Clínica	Cadastro e gestão dos veterinários: nome, CRMV único (normalizado para `CRMV-UF 12345`), especialidade, contato e situação ativo/inativo. Veterinário inativo não recebe consultas.
REQ 07	Agendar Consulta	Agenda	Agendamento relacionando animal, tutor, veterinário, data e horário. Só no futuro, de segunda a sábado das 8h às 18h, na grade de 30 min. Impede sobreposição para o mesmo veterinário e para o mesmo animal, com restrição única no banco contra dupla marcação simultânea. Mostra os horários livres do veterinário e lista as consultas do dia.

Requisitos fora desta sprint

REQ 03 (Recuperar Senha), 08 (Cancelar Consulta), 09 (Consultar Agenda por dia/semana/profissional), 10 (Registrar Atendimento), 11 (Manter Prontuário), 12 (Registrar Vacinação), 13 (Emitir Prescrição), 14 (Registrar Pagamento), 15 (Dashboard Financeiro), 16 (Notificar Tutor) e 17 (Relatórios Gerenciais). A listagem simples das consultas do dia foi incluída como apoio ao REQ 07 (para conferir o agendamento); as visões por semana e por profissional do REQ 09 ficam para a próxima sprint.

3. Arquitetura implementada

O estilo da Etapa 1 foi mantido: uma única aplicação implantável, dividida em módulos por domínio. Cada módulo tem `dominio` (entidades, regras e portas, sem banco nem web), `aplicacao` (serviços que coordenam os casos de uso) e `adaptadores` (repositório SQLite e controlador HTTP), e expõe apenas um arquivo `api.py` como interface pública. A figura 1 mostra o que foi implementado nesta sprint.

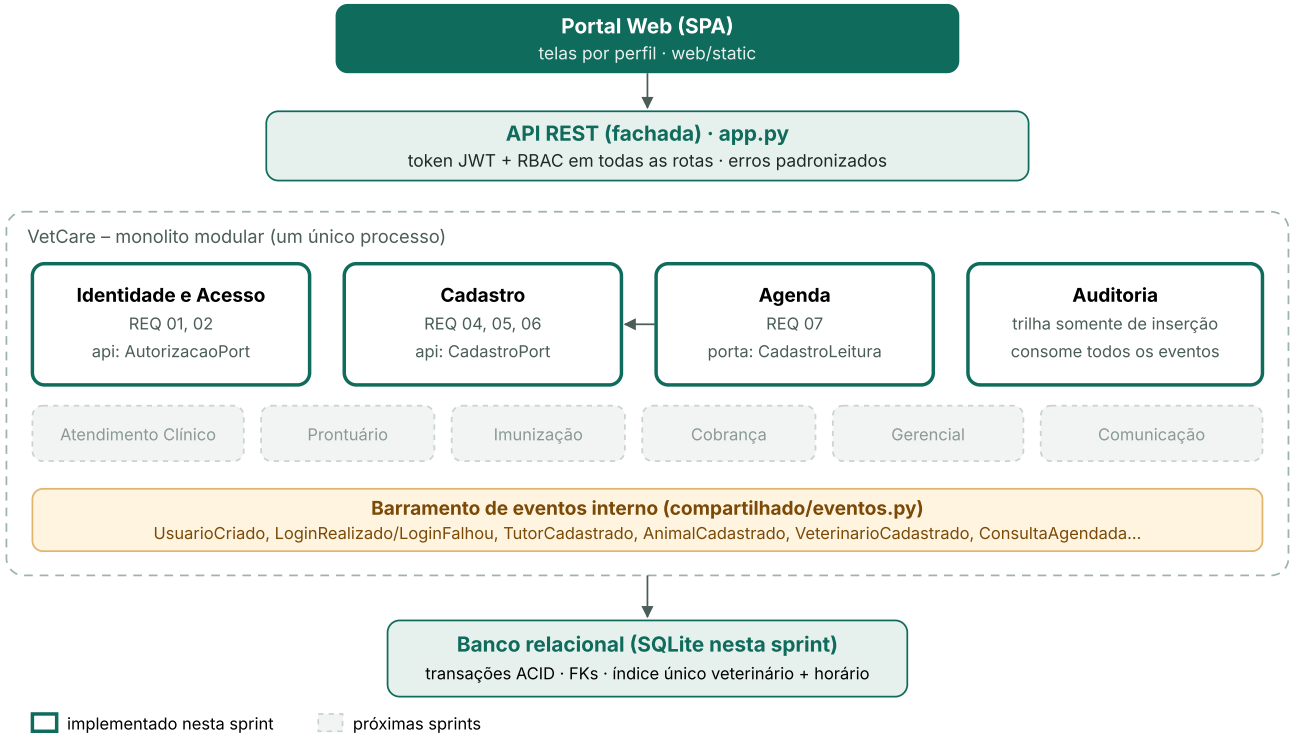


Figura 1 – Componentes da Etapa 1 implementados na Sprint 1.

3.1 Correspondência entre a Etapa 1 e o código

Elemento da Etapa 1	Onde está no código
API REST (fachada)	`vetcare/app.py`: converte as rotas dos módulos para o Starlette, valida o token e a permissão em todas as rotas e transforma exceções de domínio em erros JSON padronizados.
Identidade e Acesso (fig. 2)	`identidade/`: `AuthController` e `UsuarioController` em `adaptadores/http.py`; `AutenticacaoService`, `UsuarioService`; `PoliticaDeAcesso` (RBAC) em `dominio/politica_acesso.py`; adaptadores de hash, token e repositório; `AutorizacaoPort` em `api.py`.
Agenda (fig. 3)	`agenda/`: `AgendamentoService`, `DisponibilidadeService` (conflito de horário, regra pura), `ConsultaRepository` com índice único, publicação do evento `ConsultaAgendada`, porta consumida `CadastroLeitura`.
Tutores e Animais, Equipe Clínica	`cadastro/`: entidades `Tutor`, `Animal`, `Veterinario`; serviços; repositórios; interface pública `CadastroPort`.
Auditoria (ADR-008)	`auditoria/modulo.py`: inscrito em todos os eventos; tabela protegida contra UPDATE e DELETE por gatilhos; consulta só pelo Administrador.

compartilhado / infraestrutura	Eventos, exceções e tipos comuns; conexão com o banco, configuração externalizada e contrato HTTP neutro.
Portal Web	`web/static/`: SPA com menu lateral e cabeçalho com o usuário logado, como nos protótipos; agenda do dia como tela inicial da recepção.

3.2 Táticas arquiteturais nesta sprint

Característica	Tática (Etapa 1)	Situação	Como foi feita
Segurança	Hash com sal	**Feita**	scrypt com sal de 16 bytes e parâmetros da OWASP (ADR-012).
	RBAC na fachada e na regra de negócio	**Feita**	Permissão declarada em cada rota; política centralizada; nega por padrão.
	Bloqueio após falhas e expiração de sessão	**Feita**	5 tentativas → 15 min de bloqueio; token expira em 8 h; usuário revalidado no banco a cada requisição.
	Validação de entrada e consultas parametrizadas	**Feita**	Validação no domínio; SQL sempre com parâmetros; HTML escapado na SPA; cabeçalhos de segurança (CSP, X-Frame-Options).
Integridade	Transações e restrições no banco	**Feita**	`BEGIN IMMEDIATE`, FKs com `RESTRICT`, índice único parcial veterinário + horário ativo.
	Chave de idempotência / prontuário versionado	**Próx. sprint**	Pertencem a Cobrança e Prontuário, ainda não implementados.
Manutenibilidade	Módulos por domínio, comunicação por interfaces e eventos	**Feita**	Cada módulo expõe só `api.py`; integração por eventos.
	Regra de dependência verificada na CI	**Feita**	`tests/test_arquitetura.py` + GitHub Actions.
	Configuração externalizada	**Feita**	Variáveis de ambiente (`infraestrutura/config.py`).
Disponibilidade	Verificação de saúde	**Feita**	`GET /api/saude` testa o banco.
	Degradação graciosa	**Parcial**	Falha de um consumidor de evento não derruba a operação. Backup/restauração após migrar para PostgreSQL (ADR-011).
Desempenho	Índices e paginação	**Feita**	Índices de agenda; listagens paginadas (máx. 100).
	Cache da agenda do dia	**Adiada**	Desnecessário no volume medido (ADR-014).
Auditabilidade	Trilha somente de inserção a partir de eventos	**Feita**	Gatilhos no banco bloqueiam alteração; acesso só do Administrador.

Usabilidade	Navegação consistente; agenda como tela inicial; erros padronizados	**Feita**	Mesmo layout em todas as telas; mensagens de erro em português indicando o campo.
Testabilidade	Domínio isolado; portas substituíveis por dublês	**Feita**	Regras de agenda e login testadas sem banco, com relógio e repositórios falsos.

4. ADRs: desvios e complementos à Etapa 1

As ADR-001 a ADR-009 da Etapa 1 continuam válidas, exceto nos pontos alterados abaixo. A numeração segue a partir da ADR-010. O mesmo conteúdo está no repositório, em `docs/ADRs-Etapa2.md`.

ADR	Título	Relação com a Etapa 1
ADR-010	Linguagem e framework: Python 3 + Starlette + Uvicorn	Complementa a ADR-002 (linguagem não definida)
ADR-011	SQLite nesta sprint, PostgreSQL mantido para produção	**Desvio** parcial da ADR-004
ADR-012	Hash de senha com scrypt	**Desvio** da tática "Argon2 ou bcrypt" (ADR-003)
ADR-013	Barramento síncrono; tabela Outbox adiada para o REQ 16	**Desvio** de cronograma da ADR-007
ADR-014	Cache da agenda do dia adiado	**Desvio** parcial da ADR-009
ADR-015	Agenda declara a própria porta de leitura do Cadastro	Refinamento da ADR-005

ADR-010 – Linguagem e framework: Python 3 + Starlette + Uvicorn

Status	Aceita
Contexto	A Etapa 1 definiu o estilo (ADR-002), mas não a linguagem nem o framework. A equipe é pequena (P5) e o sistema precisa ser fácil de instalar e rodar na máquina do avaliador.
Decisão	Python 3.10+ com Starlette (servidor ASGI enxuto) e Uvicorn. O Portal Web é uma SPA em HTML/CSS/JavaScript sem framework, servida pela própria aplicação. A fachada (`app.py`) traduz um contrato neutro (`Rota`, `Requisicao`, `Resposta`) para o Starlette, de modo que os módulos não importam o framework.
Alternativas	Django (impõe ORM e organização em camadas globais, contrária à hexagonal por módulo); FastAPI (boa opção, descartada para não levar o modelo do Pydantic ao domínio e reduzir dependências); SPA com React/Vue (exigiria build com Node, sem ganho para 6 telas).
Consequências	Positivas: 3 dependências de execução; domínio em Python puro; instalação com um `pip install`. Negativas: sem OpenAPI automático (endpoints documentados no README; geração de OpenAPI prevista); a SPA sem framework exige disciplina para não crescer desorganizada.

ADR-011 – SQLite nesta sprint, PostgreSQL mantido como alvo de produção

Status	Aceita – altera parcialmente a ADR-004
Contexto	A ADR-004 escolheu PostgreSQL. Para a avaliação, o sistema precisa rodar sem que o avaliador instale e configure um servidor de banco.
Decisão	Usar SQLite (biblioteca padrão do Python) nesta sprint, mantendo as garantias que motivaram a ADR-004: transações ACID (`BEGIN IMMEDIATE`), chaves estrangeiras com `ON DELETE RESTRICT`, restrição única parcial `(veterinario_id, inicio) WHERE status='AGENDADA'` e auditoria protegida por gatilhos. O acesso ao banco fica só nos adaptadores de repositório, atrás de portas; migrar para PostgreSQL é escrever novos adaptadores, sem alterar domínio nem serviços.
Alternativas	PostgreSQL via Docker desde já (mais fiel à ADR-004, porém com mais um pré-requisito para rodar o projeto).
Consequências	Positivas: o sistema roda com um comando. Negativas: SQLite serializa escritas (suficiente para dezenas de usuários, premissa P1); backup com restauração a um instante (ADR-006) só após a migração, prevista antes da implantação.

ADR-012 – Hash de senha com scrypt

Status	Aceita – altera a tática "Senha com hash e salt (Argon2 ou bcrypt)"
Contexto	Argon2 e bcrypt exigem bibliotecas com código nativo. O objetivo da tática (um vazamento do banco não expor senhas) deve ser mantido sem dependências nativas.
Decisão	Usar scrypt (`hashlib.scrypt`) com sal aleatório de 16 bytes por senha e $N=2^{17}$, r=8, p=1, o mínimo recomendado pela OWASP quando Argon2id não está disponível. Os parâmetros ficam gravados junto do hash e a comparação usa tempo constante. O adaptador implementa a porta `GeradorDeHash`; trocar para Argon2 é trocar um adaptador. Custo configurável por `VETCARE_CUSTO_HASH`.
Alternativas	Argon2id (preferível, mas com dependência nativa); PBKDF2 (padrão, porém não exige muita memória, o que facilita ataques com GPU).
Consequências	Positivas: função que exige muita memória, sem dependência extra. Negativas: cada login leva ~0,3 s e usa 128 MiB durante a verificação; os parâmetros precisam ser revistos com o tempo.

ADR-013 – Barramento de eventos síncrono; tabela Outbox adiada para o REQ 16

Status	Aceita – adia parte da ADR-007
Contexto	A ADR-007 prevê Outbox para notificar o tutor com retentativa. Nesta sprint ainda não há notificações (REQ 16) e o único consumidor de eventos é a Auditoria.
Decisão	Implementar o barramento interno com entrega síncrona dentro da mesma transação do fato de origem; uma falha no consumidor não derruba quem publicou. A Agenda já publica `ConsultaAgendada`. Quando o módulo Comunicação for criado, ele gravará esses eventos na Outbox e um processo separado fará o envio, como na ADR-007.
Alternativas	Criar a Outbox já agora, sem nenhum consumidor (código sem uso nesta sprint).
Consequências	Positivas: auditoria gravada atomicamente com a operação; nenhum código morto. Negativas: um consumidor lento atrasaria a operação – aceitável enquanto o único consumidor é uma inserção local.

ADR-014 – Cache da agenda do dia adiado

Status	Aceita – adia parte da ADR-009
Contexto	A ADR-009 previa cache de curta duração para a agenda do dia. A equipe mediu o desempenho com cerca de 12.850 consultas (10 meses de agenda cheia para 5 veterinários) e 300 tutores: a listagem de um dia com 50 consultas levou 1,4 ms (p95) e o cálculo de horários livres 0,1 ms (p95), medidos na camada de aplicação. A meta da Etapa 1 é 2 s para 95% das consultas.
Decisão	Não implementar cache nesta sprint. Manter índices em `(veterinario_id, inicio)`, `(animal_id, inicio)` e `(inicio)` e paginação. Reavaliar com medição quando o REQ 09 (agenda por semana/profissional) for implementado.
Alternativas	Implementar o cache agora, como previsto.
Consequências	Positivas: sem risco de agenda desatualizada e sem lógica de invalidação. Negativas: se a carga crescer, o cache volta a ser necessário (pode entrar como adaptador da porta de leitura).

ADR-015 – Agenda declara a própria porta de leitura do Cadastro

Status	Aceita – refina a ADR-005
Contexto	Na Etapa 1, a Agenda consome a `CadastroPort`. Se o núcleo da Agenda importasse a classe do Cadastro, o domínio da Agenda dependeria de outro módulo.
Decisão	O núcleo da Agenda declara a porta `CadastroLeitura` (com `animal(id)` e `veterinario(id)`). O Cadastro publica `CadastroPort` em `cadastro/api.py` e o ponto de composição liga uma à outra. Testes de arquitetura verificam que o domínio não importa infraestrutura, que módulos só usam o `api.py` uns dos outros, que as dependências seguem a tabela da Etapa 1 e que não há ciclos.
Alternativas	Importar `CadastroPort` diretamente no serviço da Agenda.
Consequências	Positivas: a Agenda é testada com um Cadastro falso; a dependência Agenda→Cadastro existe em tempo de execução, como no diagrama, sem acoplamento de código. Negativas: uma interface a mais para manter.

5. Implementação

5.1 Tecnologias

Camada	Tecnologia
Linguagem	Python 3.10+
Servidor web / API	Starlette + Uvicorn (ASGI)
Banco de dados	SQLite (módulo `sqlite3`), modo WAL, FKs ativas
Autenticação	Token JWT (PyJWT, HS256) e hash scrypt (`hashlib`)
Interface	SPA em HTML, CSS e JavaScript, sem framework
Testes / CI	pytest; GitHub Actions

5.2 Estrutura do repositório

```
vetcare/
├── app.py              ponto de composição + fachada REST (autenticação e RBAC em todas as
rotas)
├── __main__.py         python -m vetcare  (sobe o servidor)
├── dados_exemplo.py    python -m vetcare.dados_exemplo  (dados de demonstração)
├── compartilhado/      eventos de domínio, exceções, tipos e validações comuns
├── infraestrutura/     banco, configuração, contrato HTTP neutro
├── identidade/         REQ 01, 02   dominio/ aplicacao/ adaptadores/ api.py
├── cadastro/           REQ 04-06    dominio/ aplicacao/ adaptadores/ api.py
├── agenda/             REQ 07       dominio/ aplicacao/ adaptadores/ api.py
├── auditoria/          trilha somente de inserção
└── web/static/         Portal Web (index.html, app.css, app.js)
tests/                  54 testes: domínio, serviços, API e arquitetura
docs/ADRs-Etapa2.md     ADR-010 a ADR-015
.github/workflows/      integração contínua (pytest)
```

5.3 Exemplo: regra de conflito de horário (domínio puro)

A regra de agendamento fica no domínio, sem banco nem web, e por isso é testada isoladamente. O serviço chama essa regra e grava a consulta na mesma transação; o índice único no banco é a última barreira contra dupla marcação:

```
class DisponibilidadeService:
    def validar(self, inicio, agora, consultas_do_veterinario, consultas_do_animal):
        fim = inicio + timedelta(minutes=self.duracao_min)
        if inicio <= agora:
            raise DadosInvalidos("Não é possível agendar consultas no passado.")
        ...  # grade de 30 min e horário de funcionamento
        if any(c.ativa and c.sobrepoe(inicio, fim) for c in consultas_do_veterinario):
            raise Conflito("O veterinário já possui consulta neste horário.")
        if any(c.ativa and c.sobrepoe(inicio, fim) for c in consultas_do_animal):
            raise Conflito("Este animal já possui consulta neste horário.")
```

5.4 Exemplo: RBAC centralizado

```
PERMISSOES = {
    "usuarios:gerenciar":    {ADMINISTRADOR},
    "tutores:ler":           {ADMINISTRADOR, RECEPCIONISTA, VETERINARIO},
    "tutores:escrever":      {ADMINISTRADOR, RECEPCIONISTA},
    "consultas:agendar":     {ADMINISTRADOR, RECEPCIONISTA},
    "auditoria:ler":         {ADMINISTRADOR},
    ...
}
# Cada rota declara sua permissão; a fachada verifica antes de chamar o módulo:
Rota("POST", "/api/consultas", agendar, "consultas:agendar")
```

5.5 Endpoints da API

Método	Rota	Permissão	REQ
POST	`/api/auth/login`	pública	02
GET	`/api/auth/eu`	autenticado	02
GET, POST, PUT, DELETE	`/api/usuarios[/{id}]`	Administrador	01
GET, POST, PUT, DELETE	`/api/tutores[/{id}]`	ler: todos · escrever: Adm., Recepção	04
GET, POST, PUT, DELETE	`/api/animais[/{id}]`	ler: todos · escrever: Adm., Recepção	05
GET, POST, PUT, DELETE	`/api/veterinarios[/{id}]`	ler: todos · escrever: Administrador	06
POST	`/api/consultas`	Adm., Recepção	07
GET	`/api/consultas?data=&veterinario_id=`	todos	07
GET	`/api/agenda/horarios-livres`	todos	07
GET	`/api/auditoria`	Administrador	–
GET	`/api/saude`	pública	–

6. Telas do sistema em funcionamento

As telas seguem os protótipos da Etapa 1: menu lateral, cabeçalho com o usuário logado e a agenda do dia como tela inicial. O menu e os botões de ação aparecem conforme o perfil. As capturas abaixo foram feitas com o sistema rodando e os dados de demonstração.

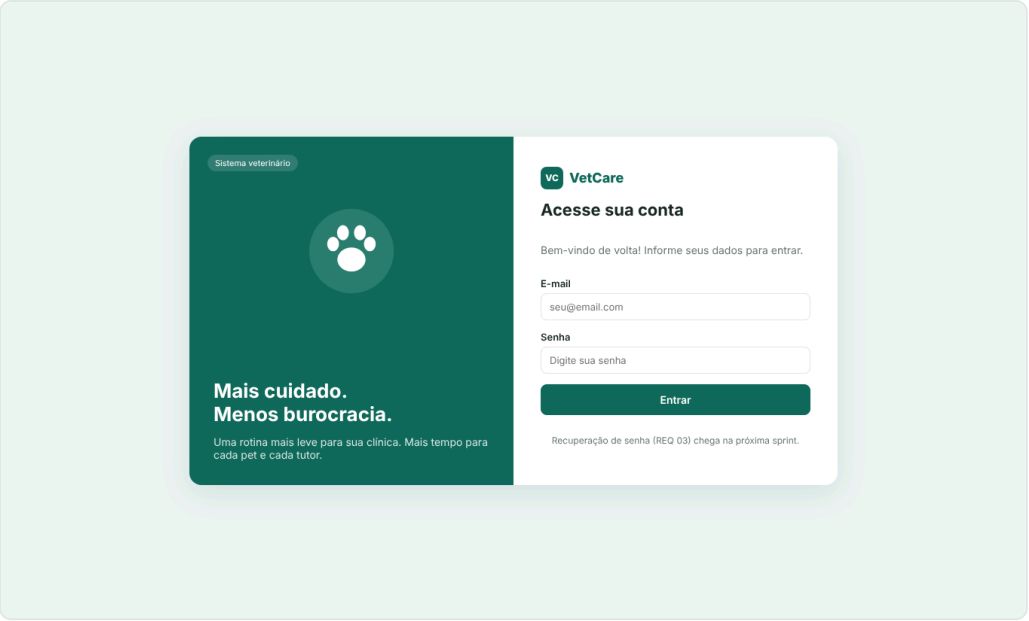


Figura 2 – Login (REQ 02).

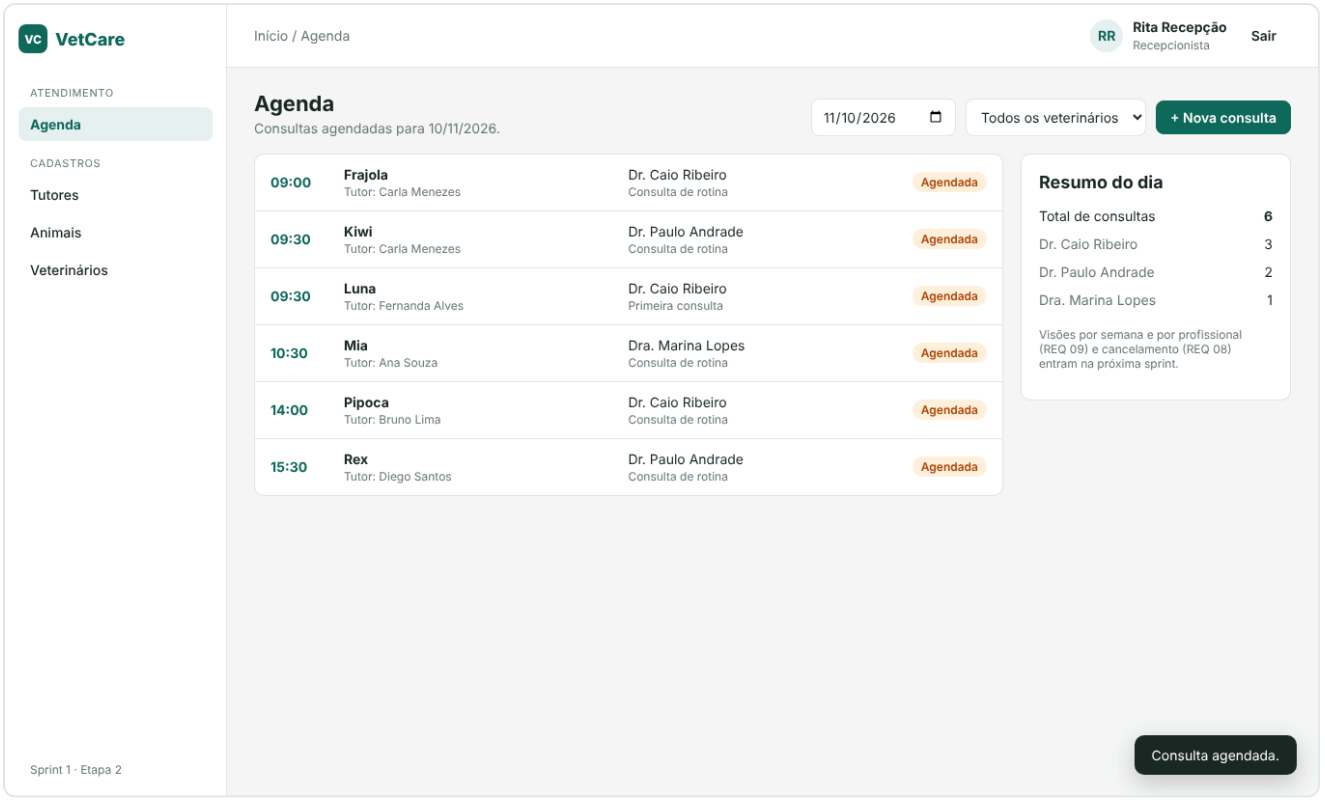


Figura 3 – Agenda do dia, perfil Recepcionista (REQ 07). A consulta da "Luna" foi criada durante o teste.

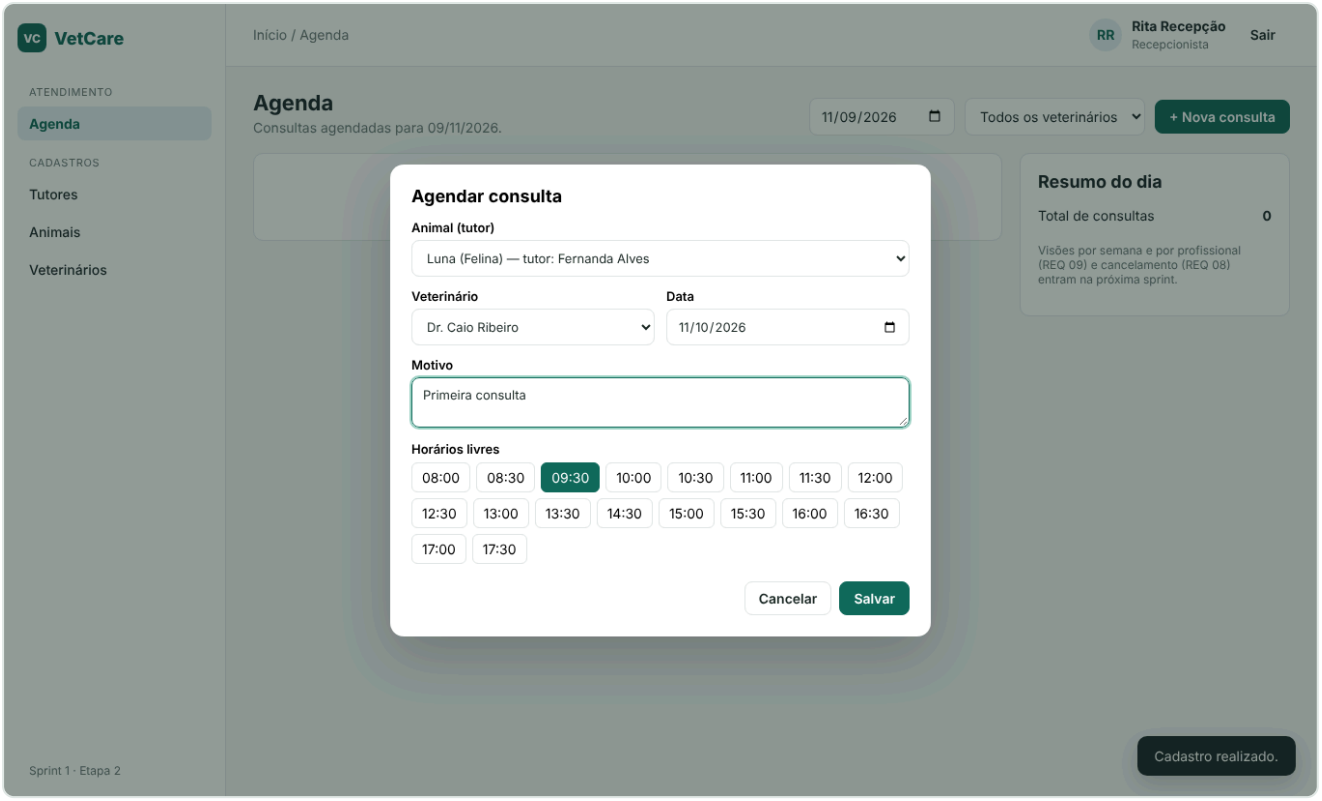


Figura 4 – Agendamento: animal (com o tutor), veterinário, data e horários livres calculados pelo sistema (REQ 07).

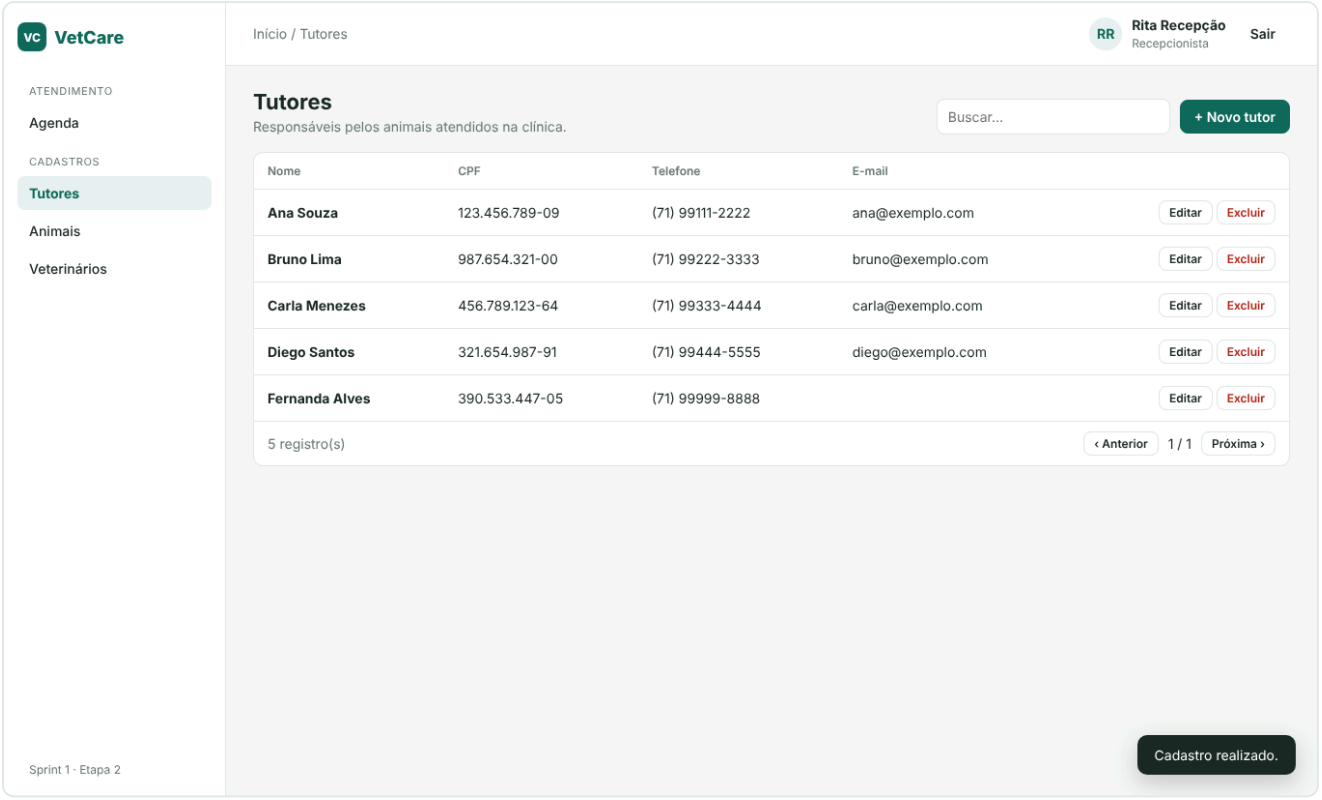


Figura 5 – Tutores, com busca e paginação (REQ 04).

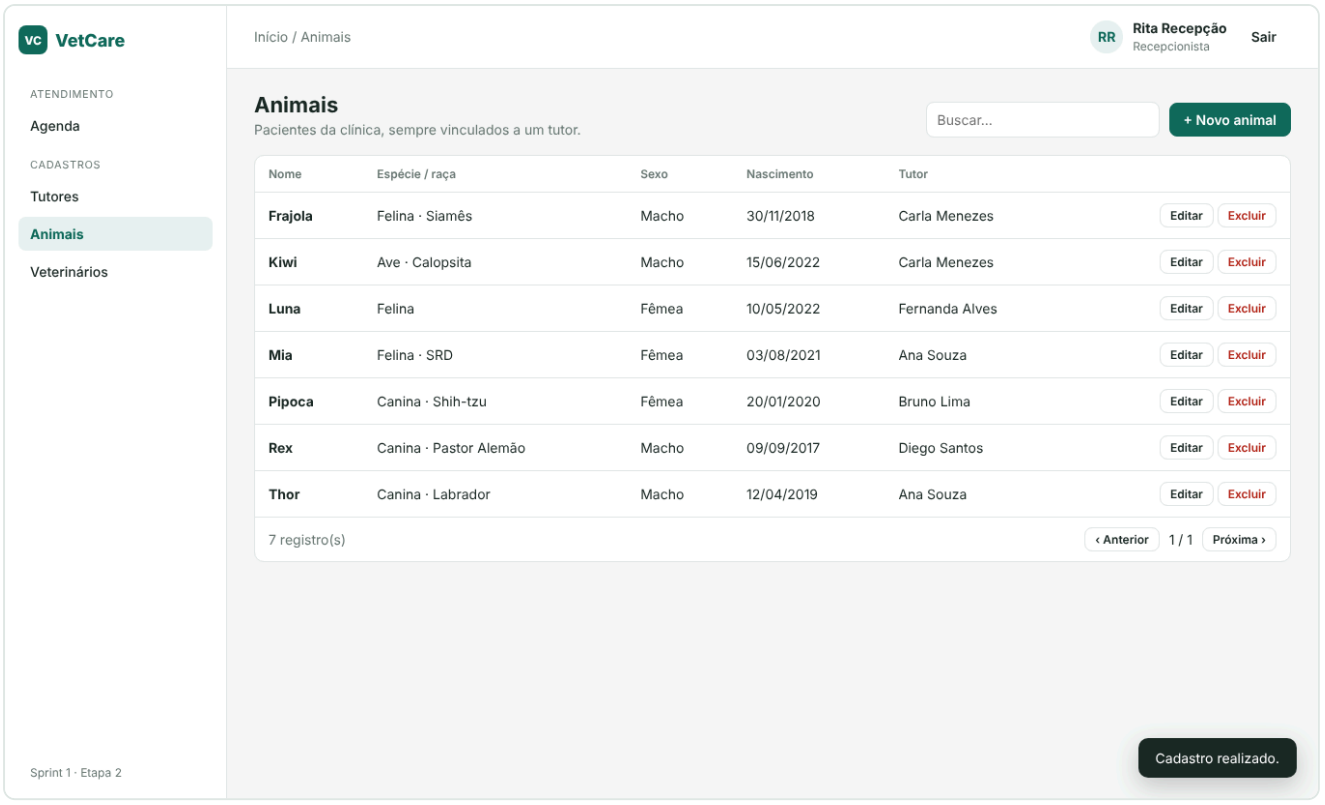


Figura 6 – Animais vinculados ao tutor (REQ 05).

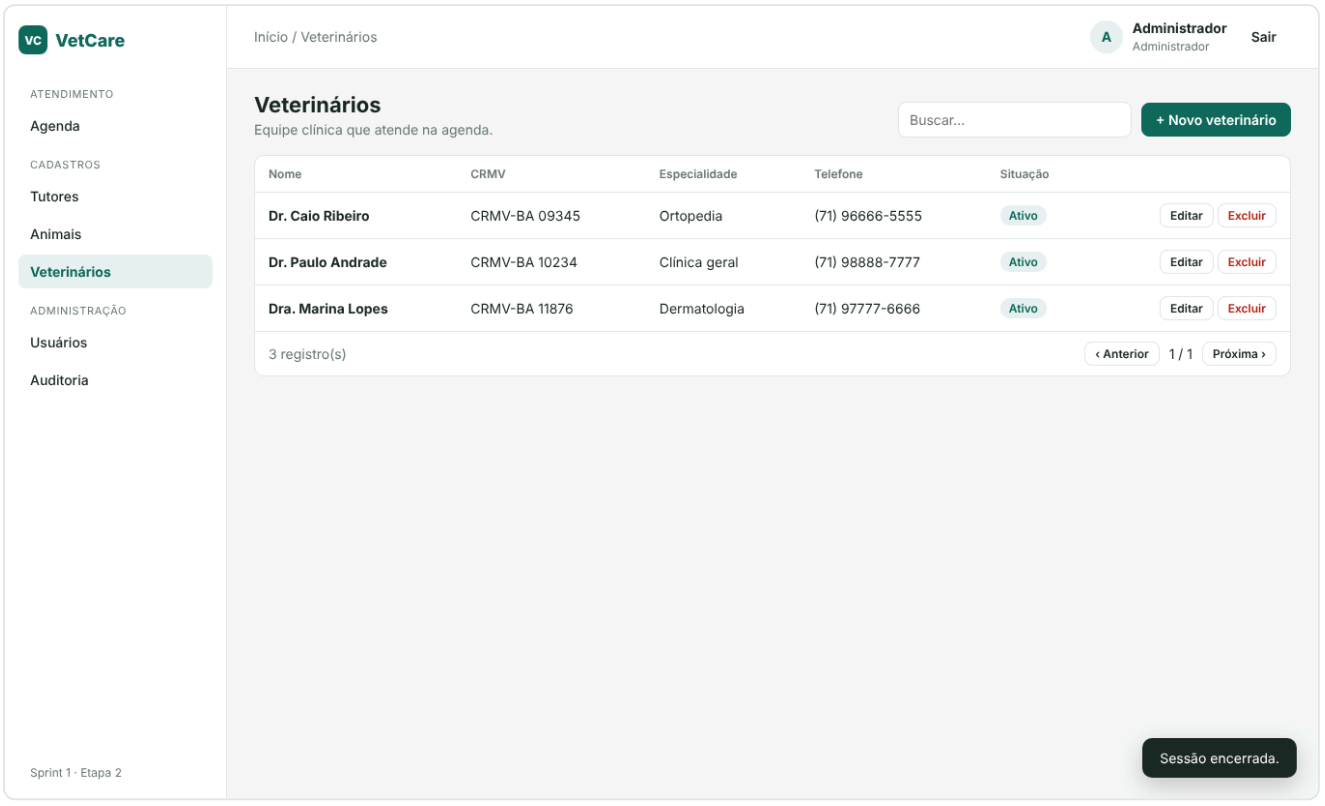


Figura 7 – Veterinários, perfil Administrador (REQ 06).

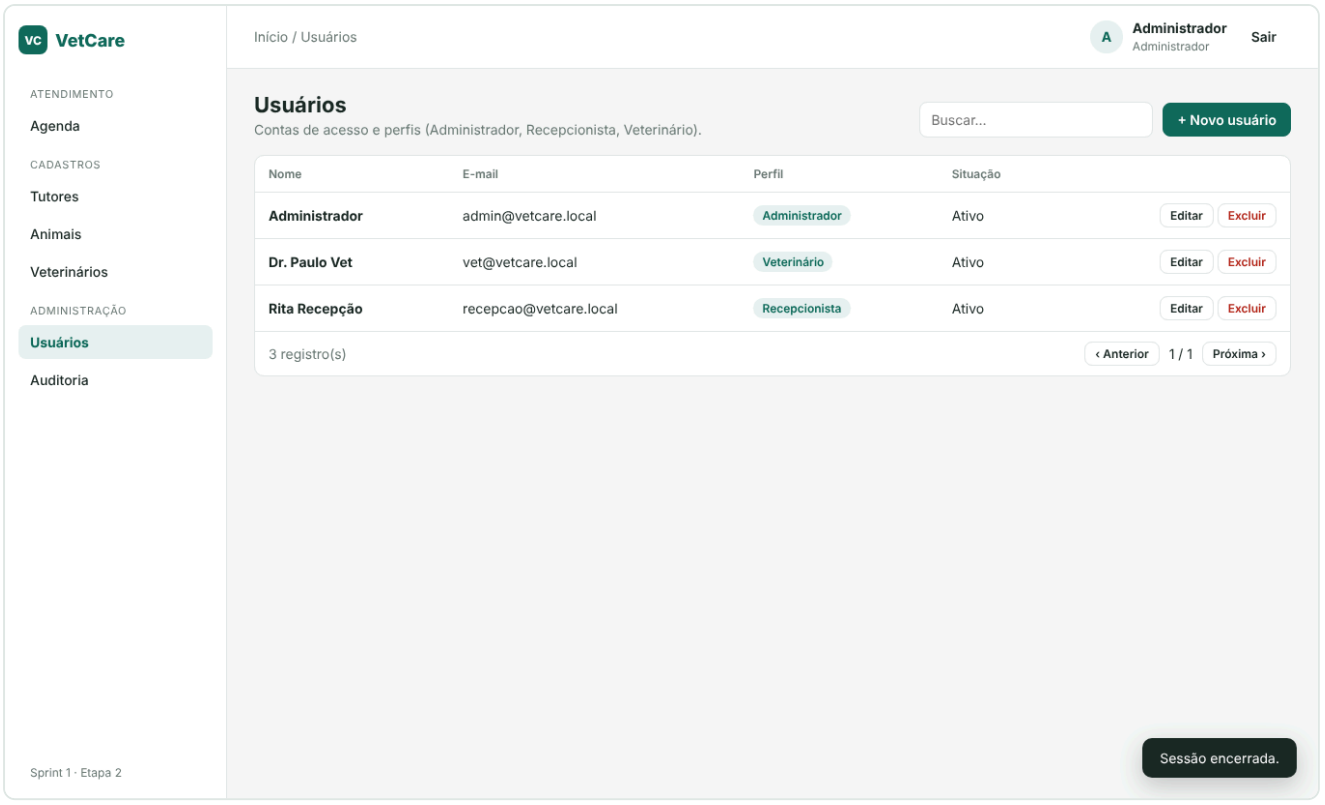


Figura 8 – Usuários e perfis, visível só para o Administrador (REQ 01).

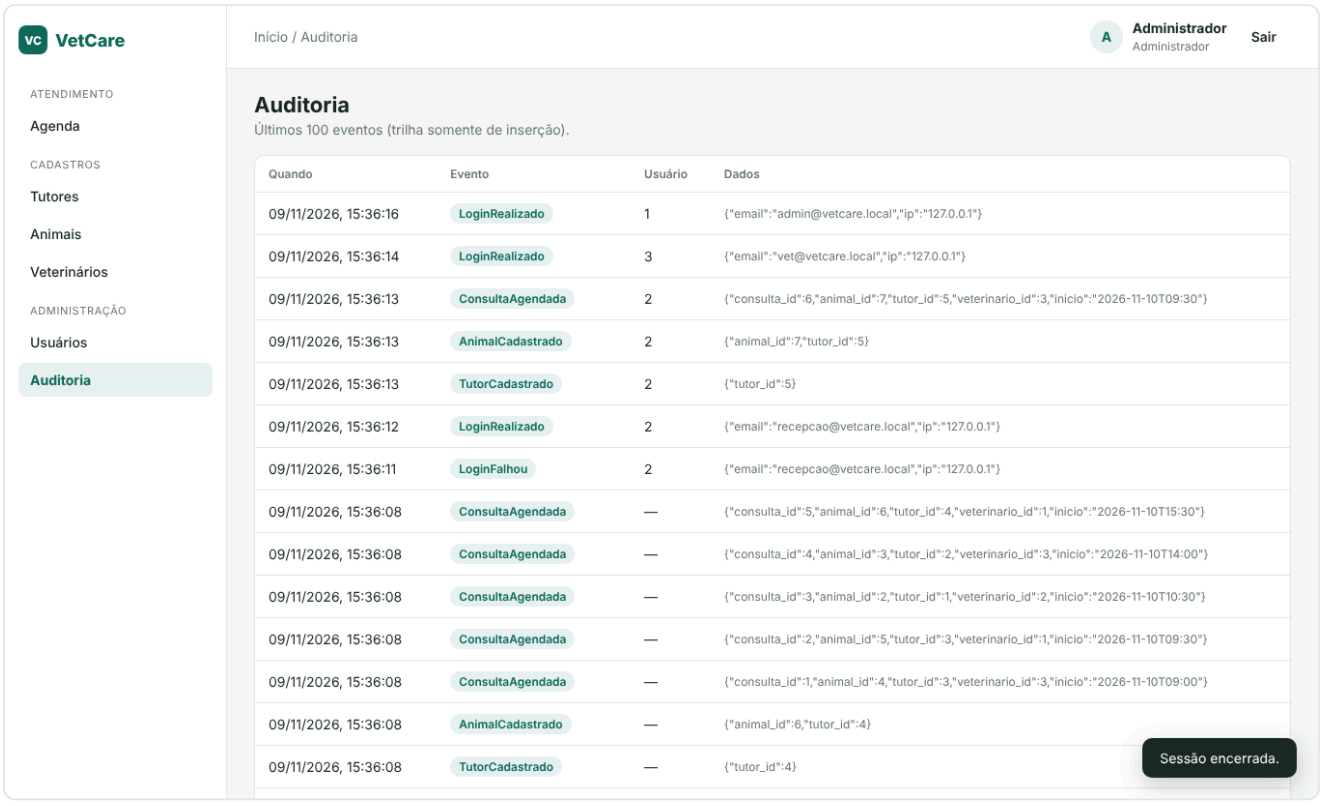


Figura 9 – Trilha de auditoria alimentada por eventos (ADR-008).

7. Testes e verificação

A suíte tem **54 testes automáticos**, todos passando, e roda em cerca de 1 segundo com `pytest`.

Arquivo	Testes	O que verifica
`test_identidade.py`	14	REQ 01 e 02 com dublês: política de senha, e-mail único, último administrador, login, mensagem genérica, bloqueio e liberação após 15 min (relógio falso), usuário desativado, token adulterado ou expirado, hash e RBAC.
`test_cadastro.py`	16	REQ 04–06: CPF (válidos e inválidos), normalização de CRMV, data de nascimento futura, CPF/CRMV duplicados, animal sem tutor, exclusão bloqueada por dependentes.
`test_agenda.py`	15	REQ 07: passado, fora do horário, domingo, grade de 30 min, conflito do veterinário e do animal, horários livres; restrição única do banco com a verificação desligada; 4 agendamentos simultâneos no mesmo horário resultam em exatamente 1 consulta.
`test_api.py`	6	Ponta a ponta pela API: 401 sem token, 403 por perfil (veterinário não cadastra tutor; recepção não vê usuários nem auditoria), fluxo completo até o agendamento, 409 em conflito, formato padronizado de erro, paginação.
`test_arquitetura.py`	3	Regras do diagrama de pacotes: domínio sem infraestrutura, acesso entre módulos só por `api.py` e conforme a tabela "Pode depender de", ausência de ciclos.

Os testes de arquitetura foram validados introduzindo uma violação de propósito (o domínio da Agenda importando `sqlite3` e uma classe interna do Cadastro): os dois testes correspondentes falharam e apontaram o arquivo e o import responsáveis.

Além da suíte, a interface foi testada no navegador com os três perfis: login com senha errada e certa, cadastro de tutor (incluindo o erro de CPF duplicado), cadastro de animal, agendamento pela escolha de horário livre, conferência dos menus por perfil e da trilha de auditoria.

8. Como executar e repositório

Repositório público: https://github.com/GuilhermePSBG/vetcare – o `README.md` traz as instruções completas.

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m vetcare.dados_exemplo    # opcional: dados de demonstração
python -m vetcare                  # http://127.0.0.1:8000
pytest                             # 54 testes
```

Perfil	E-mail	Senha
Administrador (criado na 1ª execução)	`admin@vetcare.local`	`Admin@123`
Recepcionista (dados de demonstração)	`recepcao@vetcare.local`	`Senha@123`
Veterinário (dados de demonstração)	`vet@vetcare.local`	`Senha@123`

9. Planejamento das próximas sprints

Sprint	Requisitos	Itens de arquitetura
Sprint 2	REQ 03, 08, 09, 10, 11	Módulo clínico (atendimento e prontuário somente com acréscimos e versionado); recuperação de senha com token de uso único; reavaliar cache da agenda (ADR-014).
Sprint 3	REQ 12, 13, 14, 15, 16, 17	Imunização; Cobrança com chave de idempotência e estorno; Comunicação com Outbox (ADR-007/013); Gerencial com consultas de leitura isoladas; migração para PostgreSQL e backup (ADR-011); HTTPS via proxy reverso; OpenAPI.